CS24301 — THEORY (3.0 CR) MODULE IV NOTES BIT MESRA NEP SCHEME

Module 4: Intermediate Code & Runtime

Three-Address Code (Quadruples/Triples), Arrays, Backpatching & Activation Records

Module IV: Intermediate Code Generation & Runtime Environments — Complete Topics

1. Intermediate Representations (Syntax Trees, DAG, Postfix, TAC)
2. Three-Address Code (TAC) Representations: Quadruples, Triples, Indirect Triples
3. Translation of Expressions & Type Coercion TAC
4. Multi-Dimensional Array Addressing (Row-Major vs Column-Major Proofs)
5. Boolean Expressions Translation & Short-Circuit Evaluation
6. Backpatching Techniques (`makelist`, `merge`, `backpatch`)
7. Runtime Storage Organization & Activation Records (Stack Frames)
8. Storage Allocation Strategies (Static, Stack, Heap)
9. Access Links & Displays for Nested Block Scopes
10. Parameter Passing Mechanisms & Garbage Collection

Topic 1 & 2: Three-Address Code (TAC) Representations

Three-Address Code (TAC) is a linear intermediate representation where each instruction has at most one operator on the right-hand side:

$$x = y \ \mathbf{op} \ z$$

Representation	Internal Data Structure & Fields	Key Advantages & Tradeoffs
1. Quadruples	Four fields: `(op, arg1, arg2, result)`. Explicit temporary names (e.g., `t1`, `t2`) are stored in `result`.	Easy to reorder during code optimization; requires extra memory for temporary names.
2. Triples	Three fields: `(op, arg1, arg2)`. Results of operations are referred to by their instruction index position (e.g., `(0)`, `(1)`).	Space efficient; moving or inserting instructions requires updating all index references.
3. Indirect Triples	Uses an array of pointers to separate triple structures.	Optimizers can reorder instructions by simply swapping pointers in the array without touching the underlying triples.

🏛️ Step-by-Step Solved Problem: Quadruples, Triples & Indirect Triples for `a = b * - c + b * - c`

Step 1: Generate Three-Address Code:

```
(0) t1 = uminus c
(1) t2 = b * t1
(2) t3 = uminus c
(3) t4 = b * t3
(4) t5 = t2 + t4
(5) a = t5
```

Step 2: Quadruple Representation:

#	op	arg1	arg2	result
0	uminus	c	-	t1
1	*	b	t1	t2
2	uminus	c	-	t3
3	*	b	t3	t4
4	+	t2	t4	t5
5	=	t5	-	a

Step 3: Triple Representation:

#	op	arg1	arg2
(0)	uminus	c	-
(1)	*	b	(0)
(2)	uminus	c	-
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	assign	a	(4)

Topic 4: Multi-Dimensional Array Addressing Formulations

1. 1D Array Address Formula:

$$\text{Address}(A[i]) = \text{base} + (i - \text{low}) \times w$$

Where base is the start memory address, low is lower index bound (typically 0 or 1), and w is byte width per element.

2. 2D Array Address Formula (Row-Major Order — C / C++ / Java):

$$\text{Address}(A[i_1, i_2]) = \text{base} + \Big((i_1 - \text{low}_1) \times n_2 + (i_2 - \text{low}_2)\Big) \times w$$

Where $n_2 = \text{high}_2 - \text{low}_2 + 1$ is the number of elements in each row.

3. 2D Array Address Formula (Column-Major Order — FORTRAN / MATLAB):

$$\text{Address}(A[i_1, i_2]) = \text{base} + \Big((i_2 - \text{low}_2) \times n_1 + (i_1 - \text{low}_1)\Big) \times w$$

Where $n_1 = \text{high}_1 - \text{low}_1 + 1$ is the number of elements in each column.

Topic 5 & 6: Backpatching in Boolean Expressions

Backpatching is a single-pass technique for generating intermediate code for control flow and boolean expressions without leaving unresolved forward jump target addresses.

`makelist(i)`: Creates a new list containing only jump instruction index i.
`merge(p1, p2)`: Concatenates two jump target lists p_1 and p_2.
`backpatch(p, i)`: Inserts target instruction label i as the jump destination into all instructions indexed in list p.

Topics 7 – 9: Runtime Storage Organization & Activation Records

During program execution, the operating system allocates a contiguous block of memory divided into 4 major logical segments:

Code Segment (Text): Read-only region holding compiled target machine instructions.
Static / Global Segment: Holds global variables, static constants, and compiler metadata fixed at compile time.
Call Stack: Grows downward dynamically; stores **Activation Records (Stack Frames)** for function invocations.
Heap: Grows upward dynamically; handles explicit dynamic memory allocation (`malloc`, `free`, `new`).

Activation Record Field	Functional Purpose in Procedure Call / Return
Actual Parameters	Arguments passed by the caller to the callee.
Returned Values	Space for return data passed back to caller.
Control Link (Dynamic Link)	Points to caller's activation record; used to restore caller's stack frame on return.
Access Link (Static Link)	Points to activation record of the statically enclosing lexical scope; used to access non-local variables.
Saved Machine Status	Program counter (PC), CPU registers, and status flags saved before function call.
Local Data	Local automatic variables allocated within the function.
Temporaries	Temporary values generated during expression evaluation.

🏛️ Top BIT Mesra Exam Questions & Answers (Module IV)

Q1. Compare Quadruples, Triples, and Indirect Triples with examples. (8 Marks)

1. **Quadruples:** Explicit 4-field structures `(op, arg1, arg2, res)` where temporary names (`t1`) are explicitly stated. Highly flexible for compiler optimization reordering, but uses more memory.
2. **Triples:** 3-field structures `(op, arg1, arg2)` where intermediate results are referenced by instruction array indices `(0)`. Memory compact, but relocating code requires patching all dependent instruction references.
3. **Indirect Triples:** Stores pointers to triple structures in an array. Allows optimizers to reorder and eliminate code simply by modifying pointer positions without touching underlying triple records.

Q2. Explain Call-by-Value, Call-by-Reference, and Call-by-Name parameter passing. (6 Marks)

- **Call-by-Value:** Actual parameter expression is evaluated and a local copy of the value is passed to the callee. Changes inside the function do not affect the caller's variable.
- **Call-by-Reference:** Memory address (pointer) of the actual parameter is passed. Any modification inside the function directly modifies caller's variable.
- **Call-by-Name (Algol 60):** The parameter is not evaluated upfront; instead, the parameter text is literally substituted into the function body (evaluated lazily via thunks every time it is referenced).